بِسْمِ ٱللَّهِ ٱلرَّحْمَٰنِ ٱلرَّحِيمِ

In the name of Allah, Most Gracious, Most Merciful

CSE 4107
Structured Programming I

Topic: Loops

Asaduzzaman Herok
Lecturer | CSE | IUT
asaduzzaman34@iut-dhaka.edu

Loops

Loop: repeatedly execute some other statement (loop body).

Follow the white board

- *Controlling Expression*: Each time loop body is executed the controlling expression is evaluated. Continues looping until controlling expression is evaluated to false.
- *Iteration*: One execution of loop body.
- *C Iterations statements*: The `while, do,` and `for` statements.

Some other statements:

- `break, continue, goto`
- Null statements`.`

Source: C Programming Modern Approach Book

The `while` Statement

Loop: repeatedly execute some other statement (loop body).

Follow the white board

```
while ( expression ) statement
```

- Evaluate the expression: is non-zero ? execute the statement, repeat.
- Common mistake on equality check `if ( var = value).`

```
while ( i < n ) i = i * 2;
```

```
while ( i > 0 ) {
    printf("T minus %d and counting \n", i);
    i--;
}
```

Source: C Programming Modern Approach Book

The `while` Statement

Infinite Loop: controlling expression always evaluate as true.

Follow the white board

```
while ( true ) statement
```

- Evaluate the expression: always true, execute the statement, infinite times.
- Stop: use `break, goto, return` statement to transfer control out of the loop
- Example:

```
while ( true ) {
    printf("T minus %d and counting \n", i);
    if ( i == 0) break;
}
```

Source: C Programming Modern Approach Book

The do Statement

Infinite Loop: controlling expression always evaluate as true.

Follow the white board

```
do statement while ( expression );
```

- Unlike `while` statement, body of `do` statement should be at least one statement.
- Controlling expression must be enclosed in parentheses.
- Loop body is executed first then controlling expression is evaluated. Repeat until controlling expression is false.
- Example: Number of digit calculation

```
do {
    statement;
    … …;
    … …;
} while ( expression );
```

Source: C Programming Modern Approach Book

The `for` Statement

Counting Loop: have a counting variable.

Follow the white board

```
for ( expr1; expr2; expr3 ) statement;
```

```
for ( i=10; i > 0; i--)
    printf("T minus %d, and counting\n", i);
```

- First: `expr1` is executed. (Only once)
- Next: `expr2` is executed for condition checking. If false go to the next statement after for.
- Next: if `expr2` is true, execute the body of the `for` loop
- Next: expr3 is executed.
- Next: `expr2` is executed for condition checking. If false go to the next statement after for.
- Next: expr3 is executed.
- … … …

Source: C Programming Modern Approach Book

The `for` Statement

Idioms: count up / count down loop

Follow the white board

- Count up from 0 to n-1:
- Count up from 1 to n:
- Count down from n-1 to 0:
- Count down from n to 1:

```
for (i=0; i < n; i++) ...;
for (i=1; i <= n; i++) ...;
for (i=n - 1; i >= 0 ; i--) ...;
for (i=n; i > 0 ; i--) ...;
```

Source: C Programming Modern Approach Book

The `for` Statement

Omitting expression: may not need all 3 expressions.

Follow the white board

```
for (; expr2; expr3 ) statement;
  for (; expr2;  ) statement;
 for ( ; ; expr3 ) statement;
   for ( ; ; ) statement;
```

- Missing first expression: initialization can be done before and outside the for loop.
- Missing third expression: body needs to be responsible for making the 2nd expression false (otherwise infinite loop)
- Missing second expression: it defaults to a true value.

Source: C Programming Modern Approach Book

The `for` Statement

Declaration of count variable: From C99 first expression can be declaration of variable with initialization.

Follow the white board

```
for (int i = 0 ; expr2; expr3 ) statement;
```

- Scope of declared `i`: The declared i variable in first expression can only be accessed from the body of the loop.

Source: C Programming Modern Approach Book

The `for` Statement

Comma Operator: Declaration two or more variable with initialization. Two or more increment/decrement operation.

Follow the white board

```
for (int i = 0, j = n ; expr2; i++, j-- ) statement;
```

- A comma expression: `expr1, expr2`
- First `expr1` is evaluated and its value is discarded.
- Second `expr2` is evaluated and its value is the value of the entire expression. `x = (a, b);` // x has value of b
- Comma operator is left associative.

```
i = 1,  j = 2,  k = i + j   //as ((i = 1),  (j = 2)),  (k = (i + j))
                              Performed left to right
```

Source: C Programming Modern Approach Book

Exiting from a Loop

Exit at some middle point: The break statement.

Follow the white board

```
for (int i = 0, j = n ; expr2; i++, j-- ) statement;
```

- Discussed how `break` statement can transfer control out of `switch` statement.
- Similar effect in case of loops.
- Example: Checking number if prime:

```
for ( int d = 2; d < n; d++ )
{
    if ( n % d == 0)
        break;
}
if ( d < n ) printf(“ Not prime \n”)
else printf(“prime \n”)
```

Source: C Programming Modern Approach Book

Exiting from a Loop

Exit at some middle point: The break statement.

Follow the white board

- Transfers control out of the innermost enclosing `while, do, for` or `switch` statement.
- For nested `switch, for, while, do` the `break` statement can only escape one level of nesting.

```
`    while (...) {
         switch (...) {
              …
              break;
              …
         }
    }
```

Source: C Programming Modern Approach Book

Skip statements in a Loop

Skip from some middle point: The `continue` statement.

Follow the white board

- `break` statement transfers control just *past* the end of the loop.
- `continue` statement transfers control to a point just *before* the end of the loop body.
- `continue` can only be used in loops
- Example: Adding only non-negative number

```
n = 0, sum = 0;
while ( n < 10 ) {
     scanf("%d", &x);
     if( x <= 0 )
          continue;
     sum += i;
     n++;
}
```

Source: C Programming Modern Approach Book

Jumping to labeled place in the program

Jump to a specific point of the program: The `goto` statement.

Follow the white board

- `break, continue` statement transfers control to other points.
- Target of `break` is the point just beyond the enclosing curly brace '}'.
- Target of `break` is the point just before the enclosing curly brace '}'.
- Target of `goto` is *any* the point provided that the point has a *label*. (`goto` and *label* needs to be in same function)
- Useful breaking nested loops

```
identifier : statement                    //Labeling


goto identifier;                          //Jumping back to the identifier
```

Source: C Programming Modern Approach Book

The Null Statement

Empty / devoid of symbol except the **;** : The null statement

Follow the white board

```
;
i = 0; ; j = 1;
```

- Good for creating loops with empty bodies.
- Example prime checking loop:

```
for ( d = 2; d < n && n % d != 0; d++)
/* empty body */  ;
```

- Also works:

```
for ( d = 2; d < n && n % d != 0; d++);
```

- Common mistakes due to null statement.

Source: C Programming Modern Approach Book

Bonus Question

```
int x = 0;

if (x < 0);
     x = 2;

while ( x < 5 );
{
     printf("Inside the while loop\n");
     x++;
}
```

How many times the printf function is going to be executed?

Source: C Programming Modern Approach Book

A sample Program

Balancing a Checkbook

Follow the white board

Source: C Programming Modern Approach Book


Q
A


THANK
YOU